



LAB SHEET- 2 NoSQL Database Design

1. Create a simple student database in an RDBMS and represent the same student data using a NoSQL document model. Compare both approaches based on schema, data storage, flexibility, and scalability.

- Create a Student table in MySQL.
- Insert at least 5 student records.
- Create an equivalent **Students** collection in MongoDB.
- Insert the same data as documents.
- Compare the two approaches.

2. Create a MongoDB database named **CollegeDB** and a collection named **Students**. Insert at least five student documents containing:

Student_ID, Name, Course, Semester, Marks

1. Display all documents.
2. Find students from a particular course.
3. Update marks of a student.
4. Delete one student.
5. Count the total number of students.

3. Create a MongoDB collection named **Orders** where each order contains customer information and its associated products as an embedded document/array.

1. Insert at least five orders.
2. Display orders of a particular customer.
3. Update the quantity of a product.
4. Calculate the total order amount using aggregation.
5. Display orders above a specified amount.

4. Using a key-value database such as Redis, create a simple student information system.

1. Store student ID as the key and student name as the value.
2. Insert at least five students.
3. Retrieve a student using the key.
4. Update a student's value.
5. Delete a student.



6. Display all available keys.

5. Create a Cassandra keyspace named **CollegeDB** and a table named **Students**.

Student_ID, Name, Course, Semester, Marks

1. Insert at least five records.
2. Display all records.
3. Retrieve students from a particular course.
4. Update the marks of a student.
5. Delete a student record.

6. Using HBase, create a table named **Students** with suitable column families such as **personal** and **academic**.

1. Insert at least five student records.
2. Retrieve a particular student's details.
3. Update student information.
4. Display all records.
5. Delete a student record.

7. Create a simple Student–Course relationship graph using Neo4j.

Create nodes for:

- Students
- Courses

Create relationships:

- **ENROLLED_IN**

Perform Cypher queries to:

1. Create at least five students and three courses.
2. Display all students.
3. Find students enrolled in a particular course.
4. Display all courses taken by a particular student.
5. Add and remove a relationship.

8. Implement the same simple student-related dataset using MongoDB, Cassandra, and Neo4j.

Compare their:

- Data model
- Storage structure
- Query method
- Scalability



- Suitable applications

9. Deploy a NoSQL database on a **single server/local machine** and perform basic database operations.

1. Install/configure MongoDB.
2. Start the database server.
3. Create a database and collection.
4. Insert sample records.
5. Perform CRUD operations.
6. Verify that all operations are working on the single server.

10. Demonstrate the concept of database sharding by distributing a large dataset across multiple logical shards.

Use a suitable NoSQL database such as MongoDB.

1. Create a sample dataset containing at least 20 records.
2. Select an appropriate shard key.
3. Configure sharding.
4. Distribute the data across shards.
5. Verify how the data is distributed.
6. Explain the benefit of sharding.

11. Demonstrate Master-Slave (Primary-Secondary) Replication using a suitable NoSQL database.

1. Configure one primary/master server.
2. Configure at least one secondary/slave server.
3. Insert data into the primary.
4. Verify that the data is replicated to the secondary.
5. Explain what happens when the primary server becomes unavailable.

12. Demonstrate the concept of Peer-to-Peer Replication using a distributed NoSQL database such as Cassandra.

1. Configure multiple Cassandra nodes.
2. Create a keyspace with an appropriate replication factor.
3. Insert sample data.
4. Verify data availability across nodes.

5. Explain how peer-to-peer architecture differs from master-slave replication.

13. Design and demonstrate a NoSQL database architecture that combines Sharding and Replication.

1. Create multiple shards.
2. Configure replication for each shard.
3. Insert sample data.
4. Verify data distribution.
5. Verify replica availability.
6. Explain how this architecture provides both scalability and fault tolerance.

14. Implement a simple MapReduce operation on a database dataset.

Use a student dataset containing:

Student_ID, Name, Course, Marks

Perform a MapReduce operation to:

1. Calculate the number of students in each course.
2. Calculate the average marks for each course.
3. Display the result of the Map and Reduce operations.

15. Study and demonstrate the following database distribution models:

1. Single Server
 2. Sharding
 3. Master-Slave Replication
 4. Peer-to-Peer Replication
 5. Sharding + Replication
- Draw the architecture.
 - Explain how data is distributed.
 - Explain how failure is handled.
 - State one advantage.
 - State one limitation.
 - Give one suitable application.